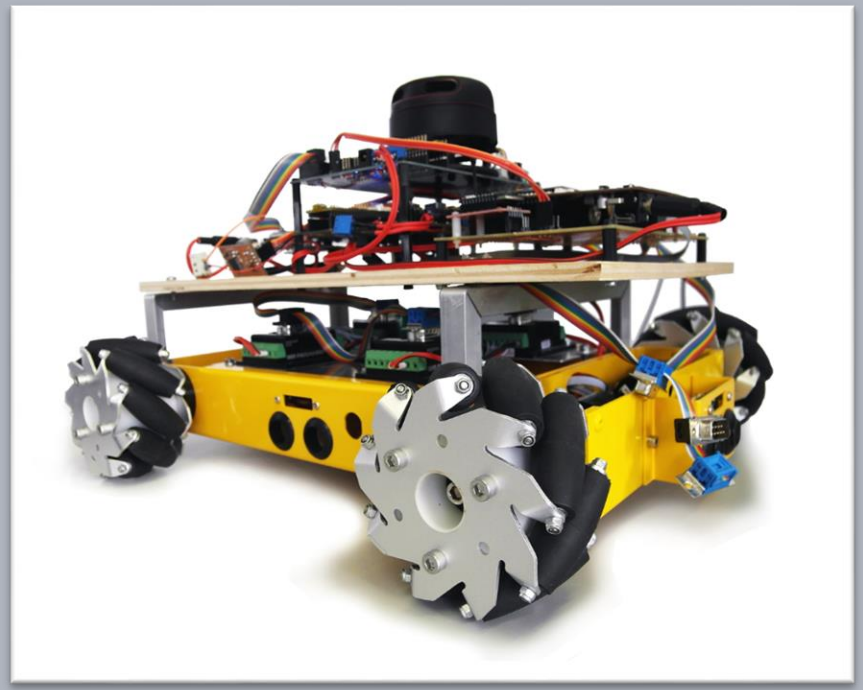
Advanced Embedded Architectures and Applications

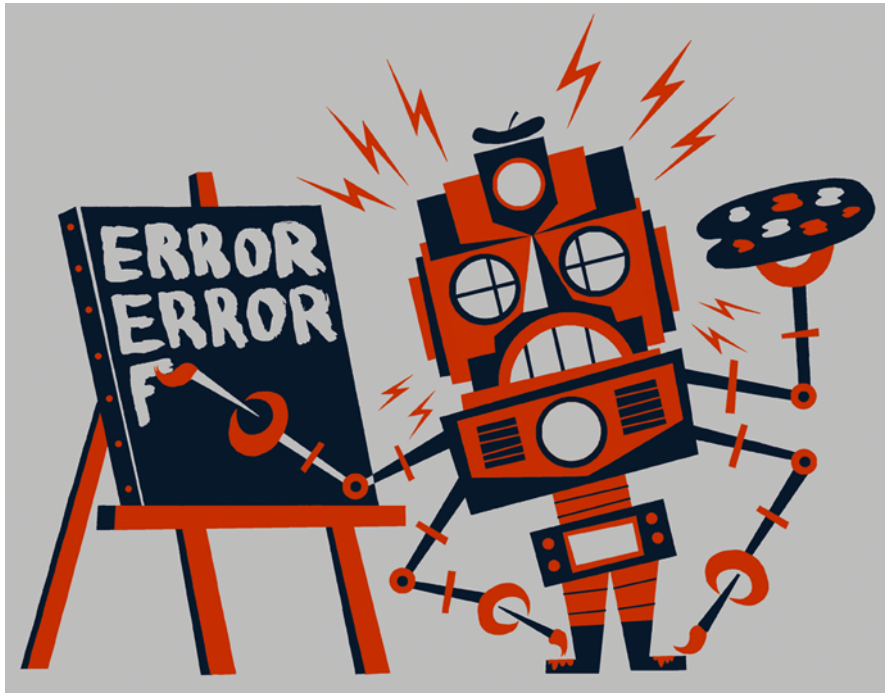
Error Handling

Prof. Dr.-Ing. Peter Fromm



Content

- Motivation and Challenge Error Handling
- Hardware / Software Interaction
- Error Detection, Handling, Escalation
- Error handling for our Car
- Error Handling Architecture
- Safe State
- Central Monitor
- Error State Machine
- Development Error Tracer



Normal versus abnormal behaviour

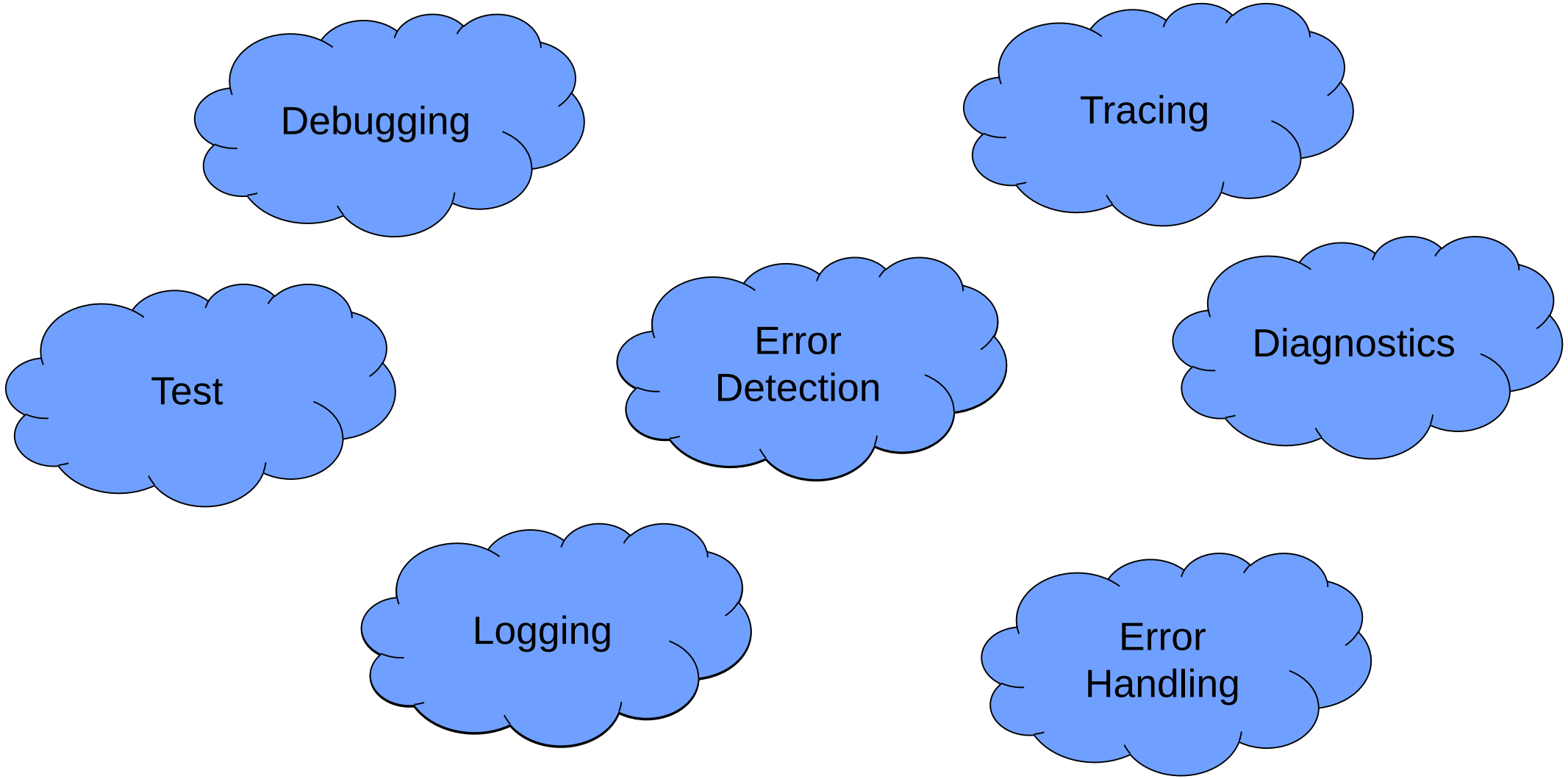
When programming an application, we typically consider the normal behaviour, which very often already is complex enough.

This however is not sufficient!

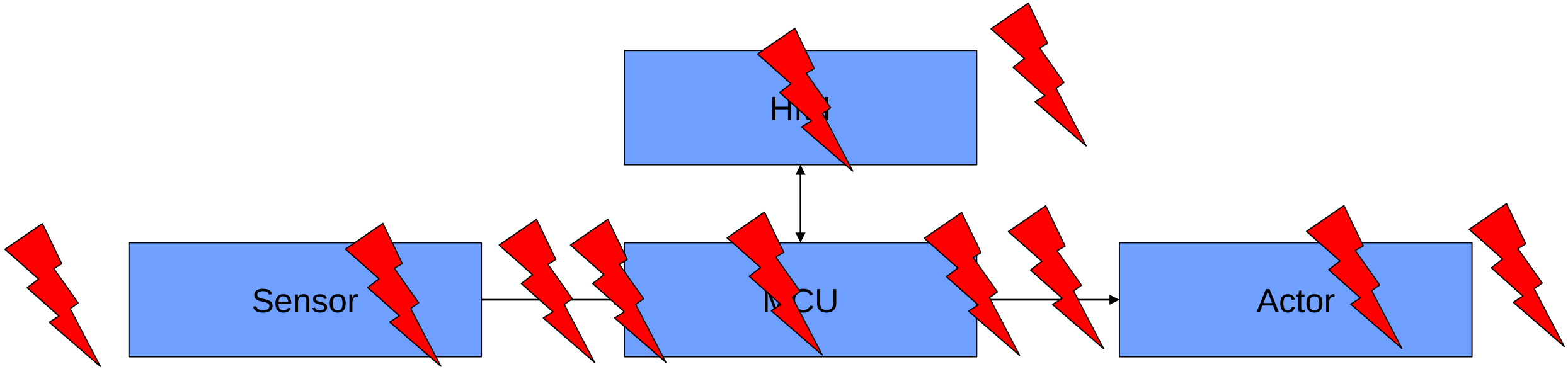
- Hardware may break
- Software may contain bugs
- Users may enter wrong data
- Communication may get lost
- Sensors may provide wrong data
- ….

Even if something unexpected happens, our system will show a well defined behaviour!

Many terms, some overlap, often confused



Chain of Problems – a bit more detailled

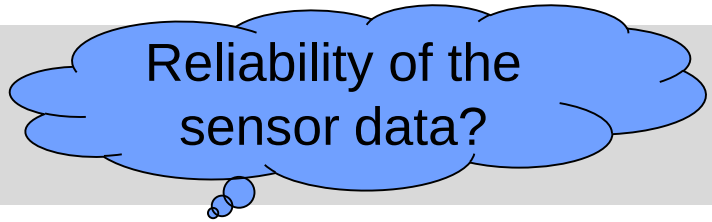


- Sensor measures wrong environmental data
- Sensor hardware broken
- Sensor communication link error
- MCU Sensor Peripheral misconfiguration

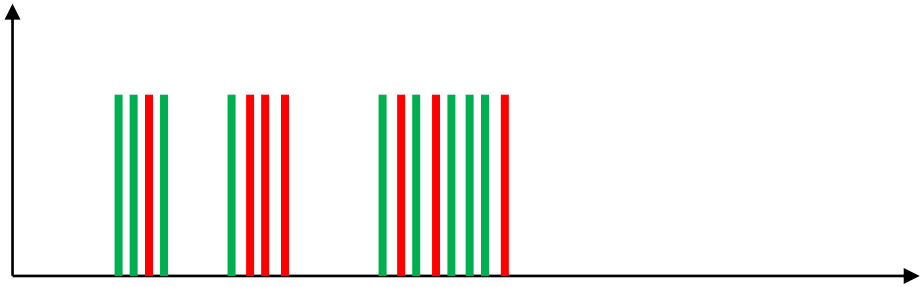
- Software Bug, OS Bug, Compiler Bug

- MCU Actor Peripheral misconfiguration
- Actor wrong action
- Actor suffers from external impact
- HMI shows wrong data
- HMI transmits wrong user command (e.g. ghost touch)
- Human misinterprets HMI data

Diagnostic Complexity. What is an error?



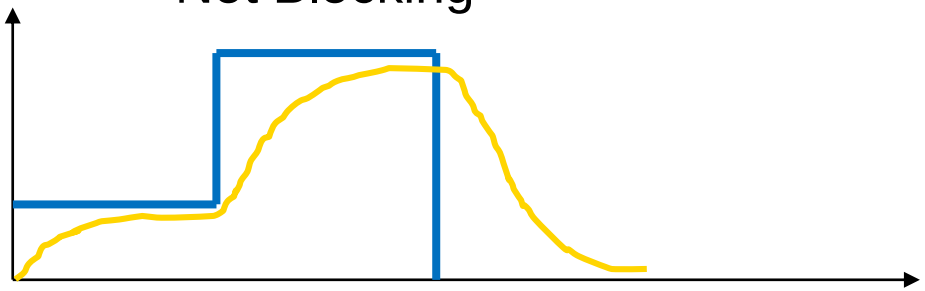
- A communication protocol has a wrong CRC
  - Might be ok, if it happens once?
  - But not after 3 times in a row?
  - Or too often in a certain interval?



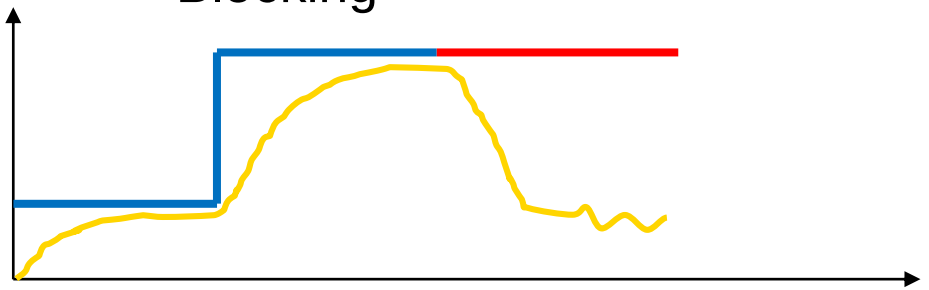
- Ultrasonic not receiving an echo
  - Could be no obstacle or a malfunction

- Detect blocking of an engine by comparing targetspeed versus currentspeed
  - Returns many wrong errors when accelerating or braking
  - We need to calculate the integrated value

Not Blocking



Blocking



Error Characteristics – ISO26262:2018 – part 8

Fault
- abnormal condition that can cause an *element* (3.41) or an *item* (3.84) to fail

Error:
- discrepancy between a computed, observed or measured value or condition, and the true, specified or theoretically correct value or condition

Failure:
- termination of an intended behaviour of an *element* (3.41) or an *item* (3.84) due to a *fault* (3.54) manifestation

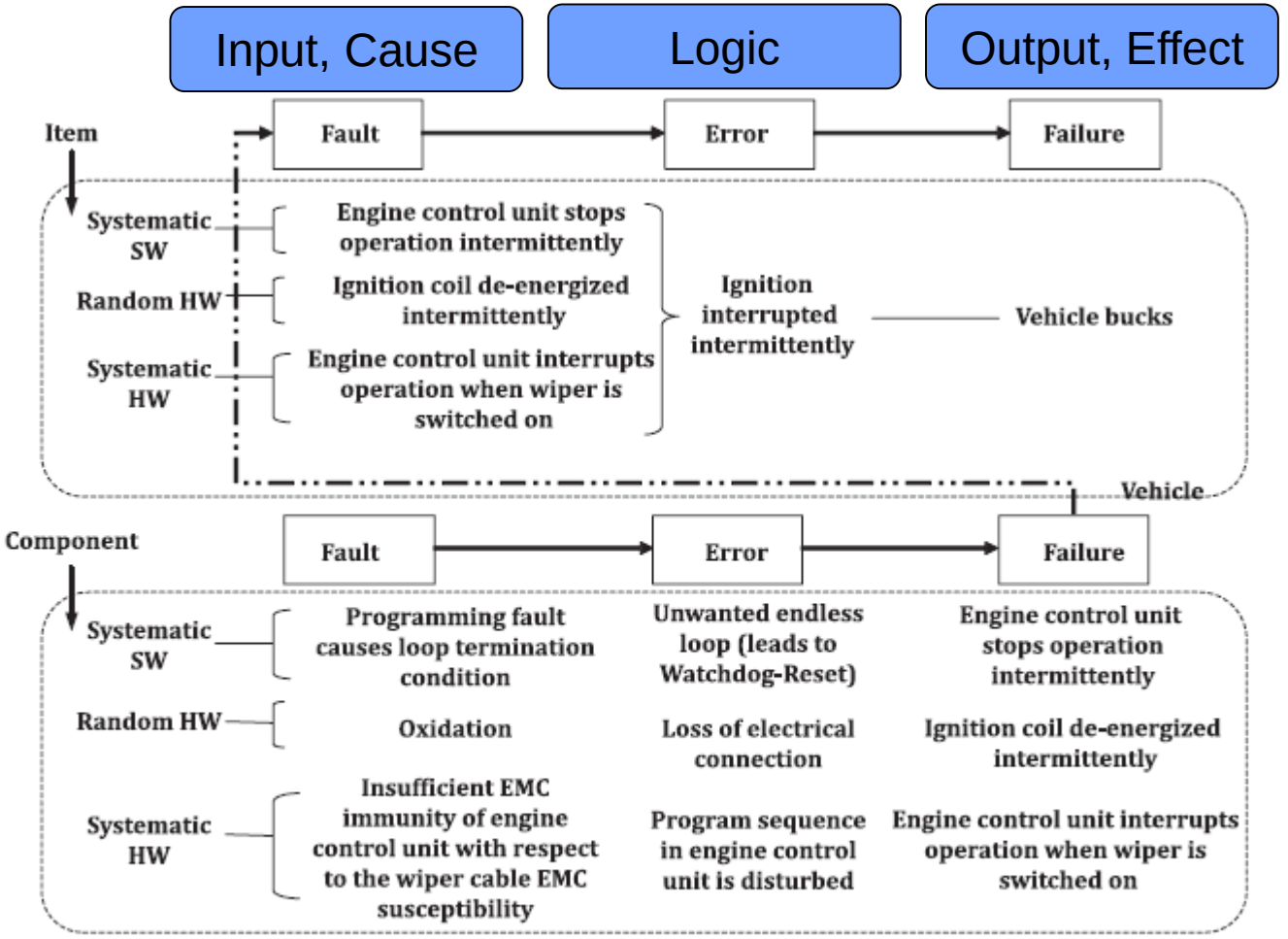
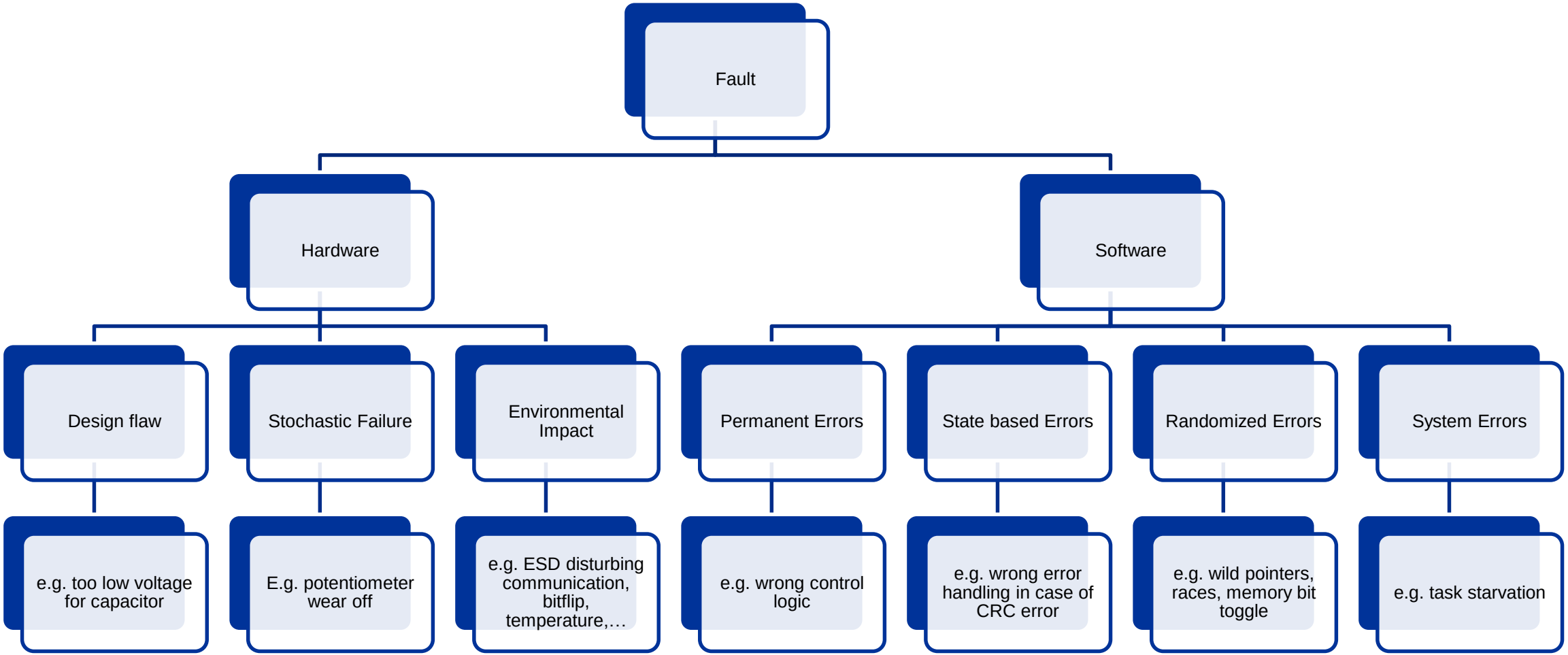
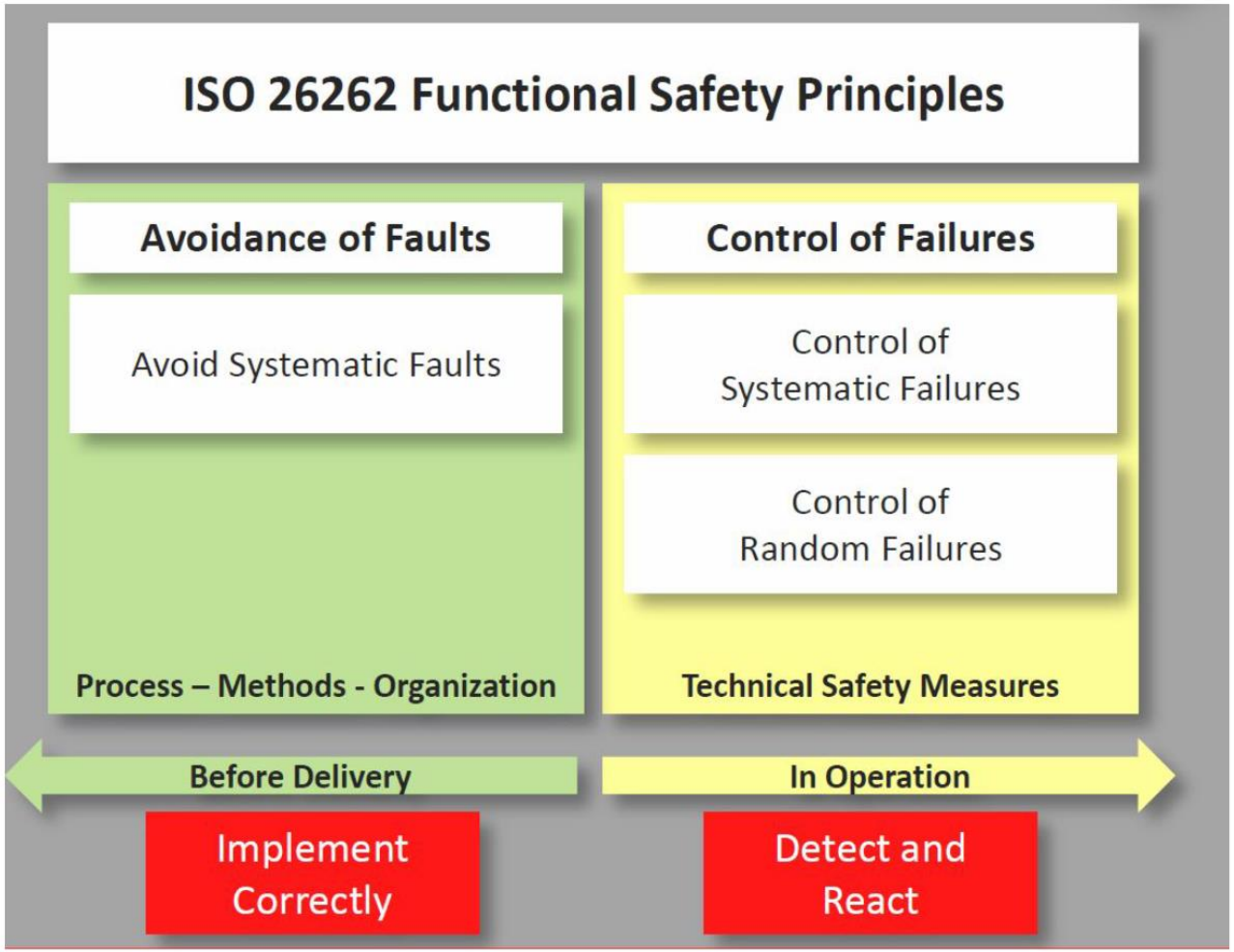


Figure 5 — Example of faults leading to failures

Error Categories (probably not complete)



Design Goal – not only for safety systems!

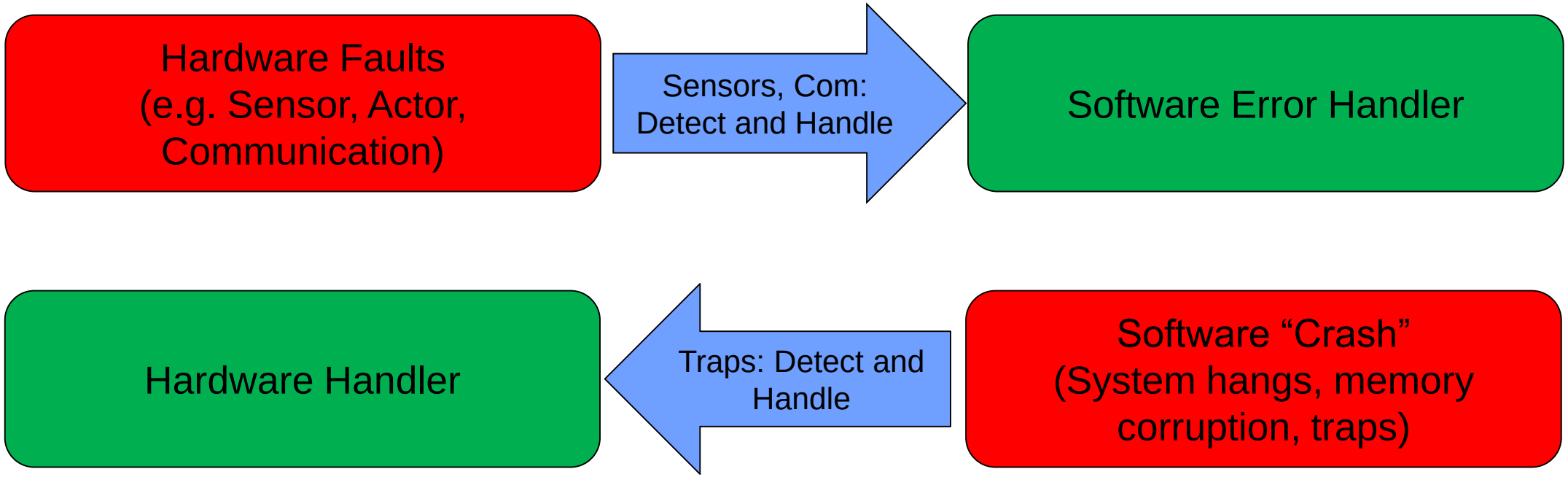


University of Toronto, CSC2125, Lecture 3: Safety Analysis

Some Recipes

Fault Category	Avoid	Detect / Handle
Systematic Hardware Failures	• Design Review • FMEA, FMEDA • Test	• Diagnostic Coverage • Redundancy • Safe State
Random Hardware Failures	• Part Selection (MTTF) • Shielding, Cooling • EMD, temperature and other environmental Tests • Analysis of field errors	
Systematic Software Failures	• Review (Testcase, Design and Code) • Static Code Analysis • Systematic test • Systematic test of abnormal (error) conditions • Stresstest	• Diagnostics • Safe design and code pattern (e.g. no endless loops,…) • Error handling architecture • Safe State
Random or State based but localised Software Failures		
System Wide Software Failures		• Memory supervision (MPU) • CPU supervision (Watchdog)

Hardware / Software Interaction

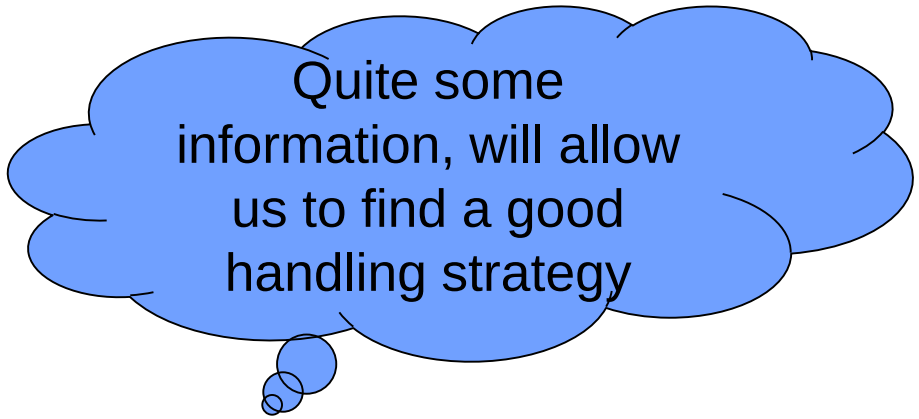


Designing a good error handling strategy means, we have to be aware of possible faults and need to develop an architecture to deal with these faults, before a failure happens.

Hardware / Software Interaction

Hardware errors handled by software

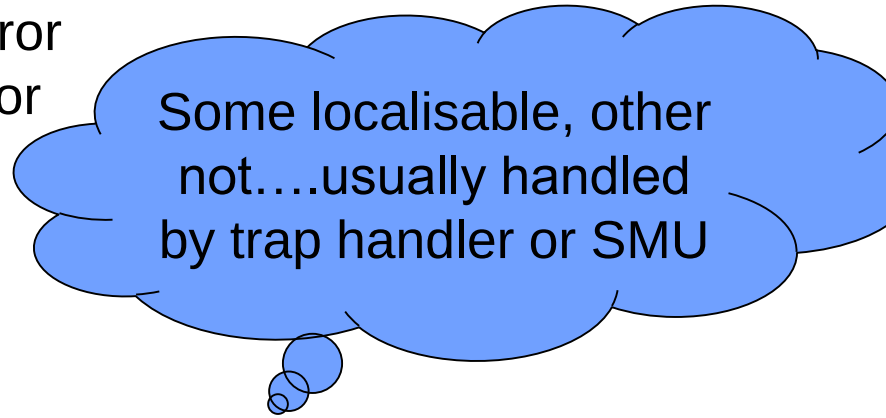
- Sensor value out of range
- Communication protocol corrupted
- Data not updated in the intended time frame



Controller error requiring escalation

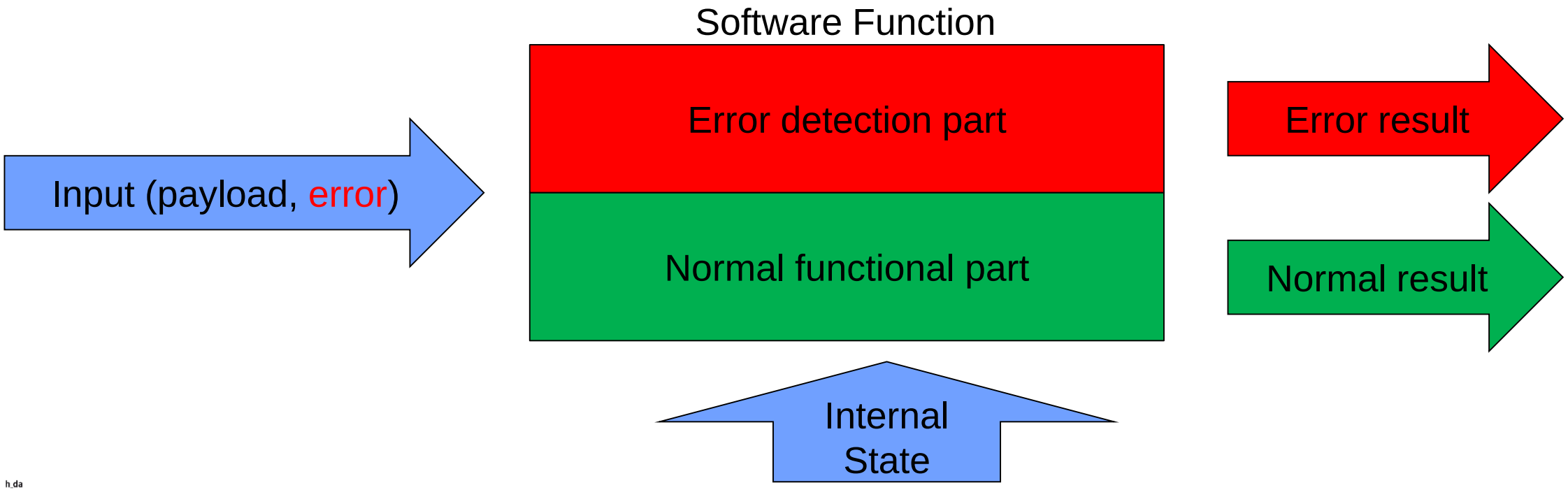
- MPU detects wrong access
- Privilege level error
- Invalid address access or alignment (e.g. void pointer)
- Call depth under/overflow

- Invalid Opcode
- Lockstep error
- Memory error



Detection versus Handling

In computing and computer programming, exception handling is the process of responding to the occurrence of exceptions – anomalous or exceptional conditions requiring special processing – during the execution of a program. In general, an exception breaks the normal flow of execution and executes a pre-registered exception handler; the details of how this is done depend on whether it is a hardware or software exception and how the software exception is implemented. (Wikipedia)



Detection versus Handling (Software still under control)

Depending on the architecture, there are different ways how error conditions are identified, handled or escalated

Detection
- Analysing the input data itself
- Analysing the validity state of input metadata (error state, age,…)
- Analysing combinations of input data and input metadata

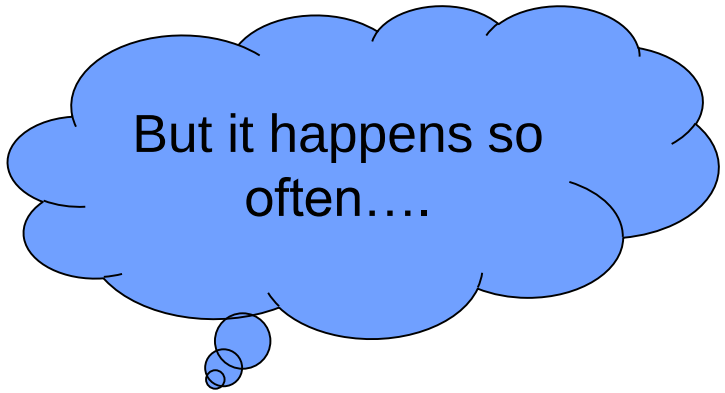
Handling
- Preferred solution – the function “knows” how to deal with the problem, e.g. using an old value for a certain time

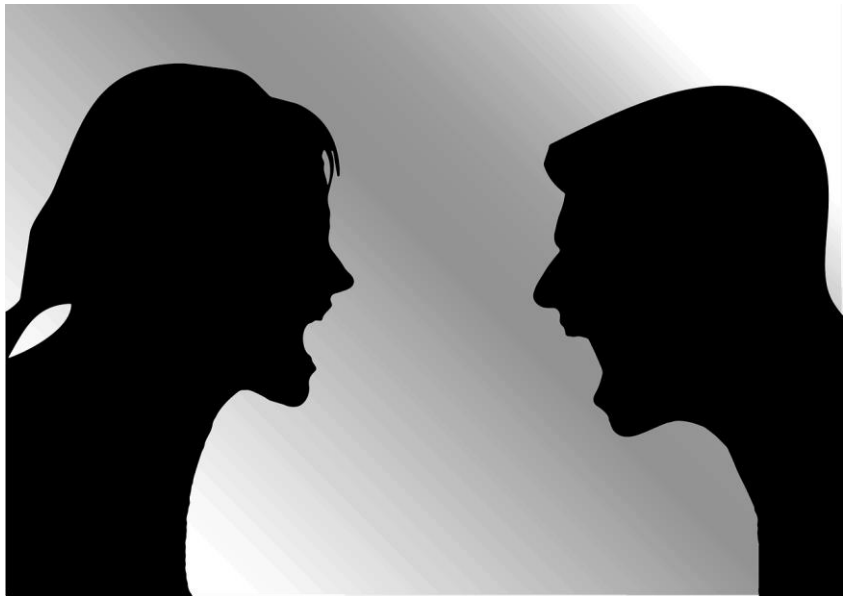
Escalation
- The function can not handle the data itself and needs to escalate the problem
- Error return value, metadata of signals, calling central error handlers,…

Error escalation

Golden rule

- Error return values indicate a missing handling inside the function
- Although the function of course must protect itself (e.g. by returning a default value instead of a measurement value)
- The problem is escalated to the caller
- Therefore, the caller NEVER may never ignore error codes!





Detection versus Handling (Software not any more under control)

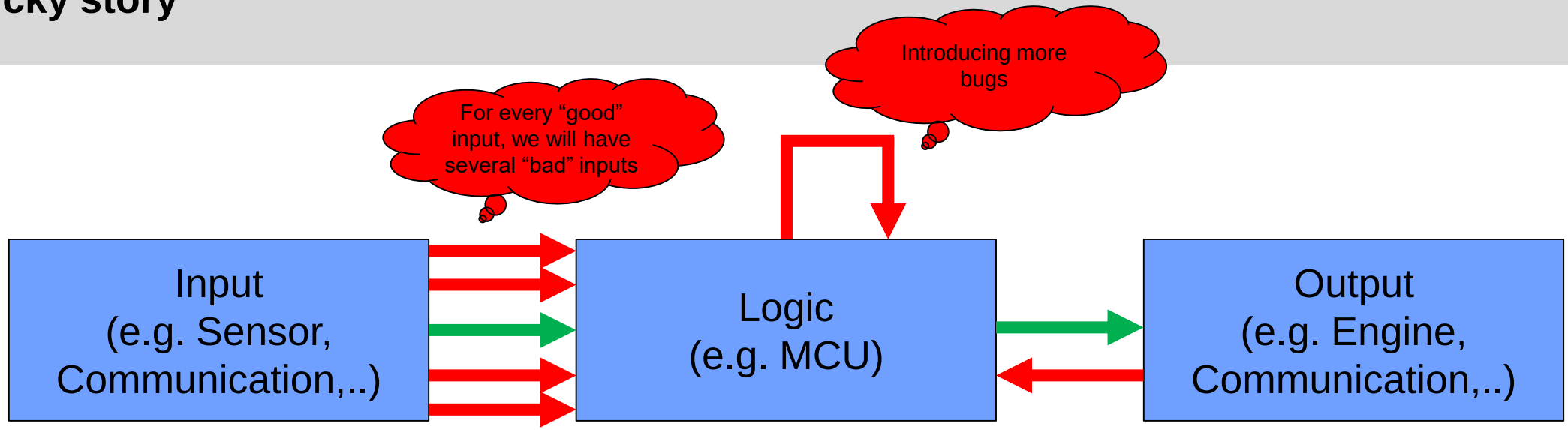
Depending on the hardware, different sources and escalation options are available.

Detection
- Trap system
- Watchdog
- Hardware components like SMU, SCO

Escalation
- Only few errors are “repairable”, e.g. memory trap ➔ reconfiguration of the MPU
- Most other errors lead to a final escalation ➔ Safe State

How about a simple Reset of the CPU – typical reaction of e.g. a watchdog error?

Tricky story

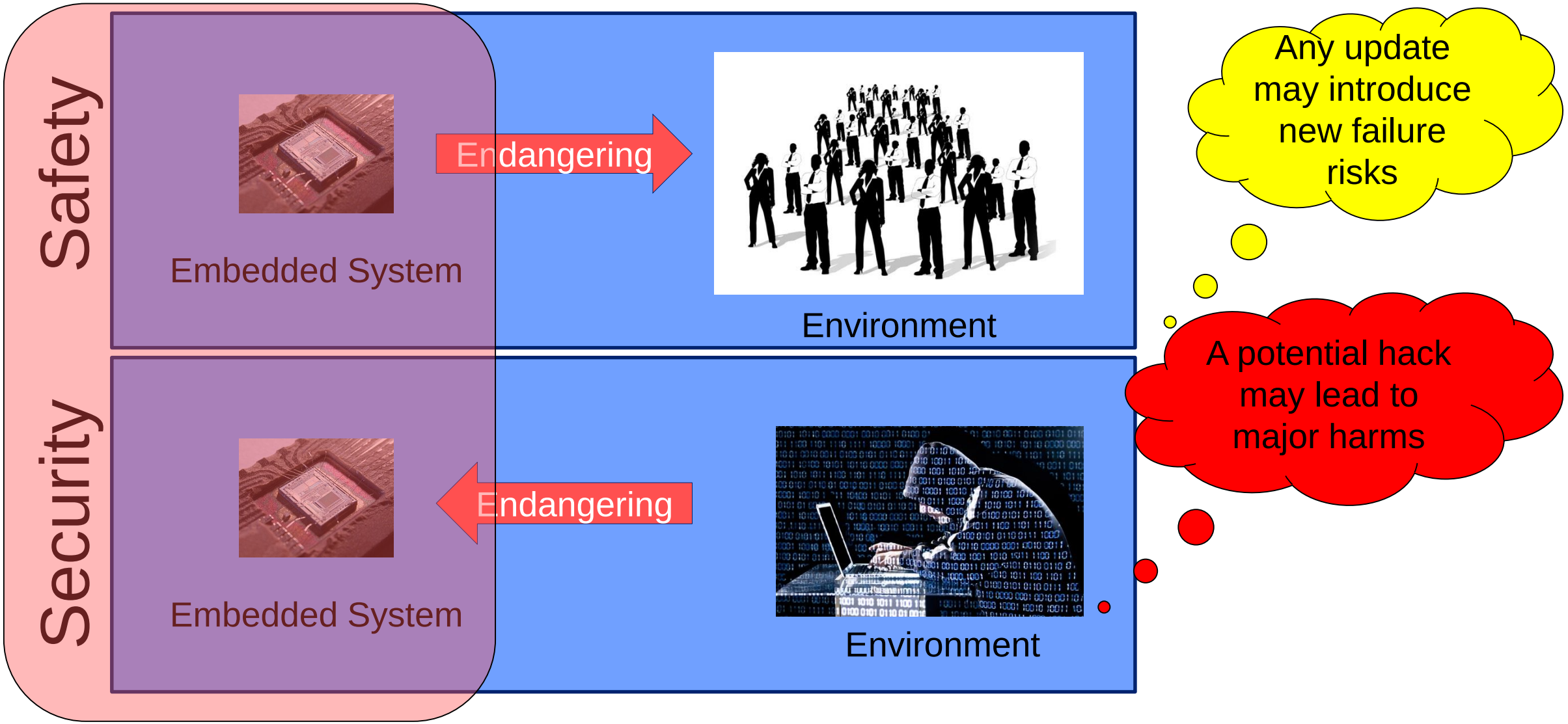


100.000 LOC

+100.000 LOC (without customer added value)

Ignoring is no option,
we have to do it!

An additional challenge arises with open systems

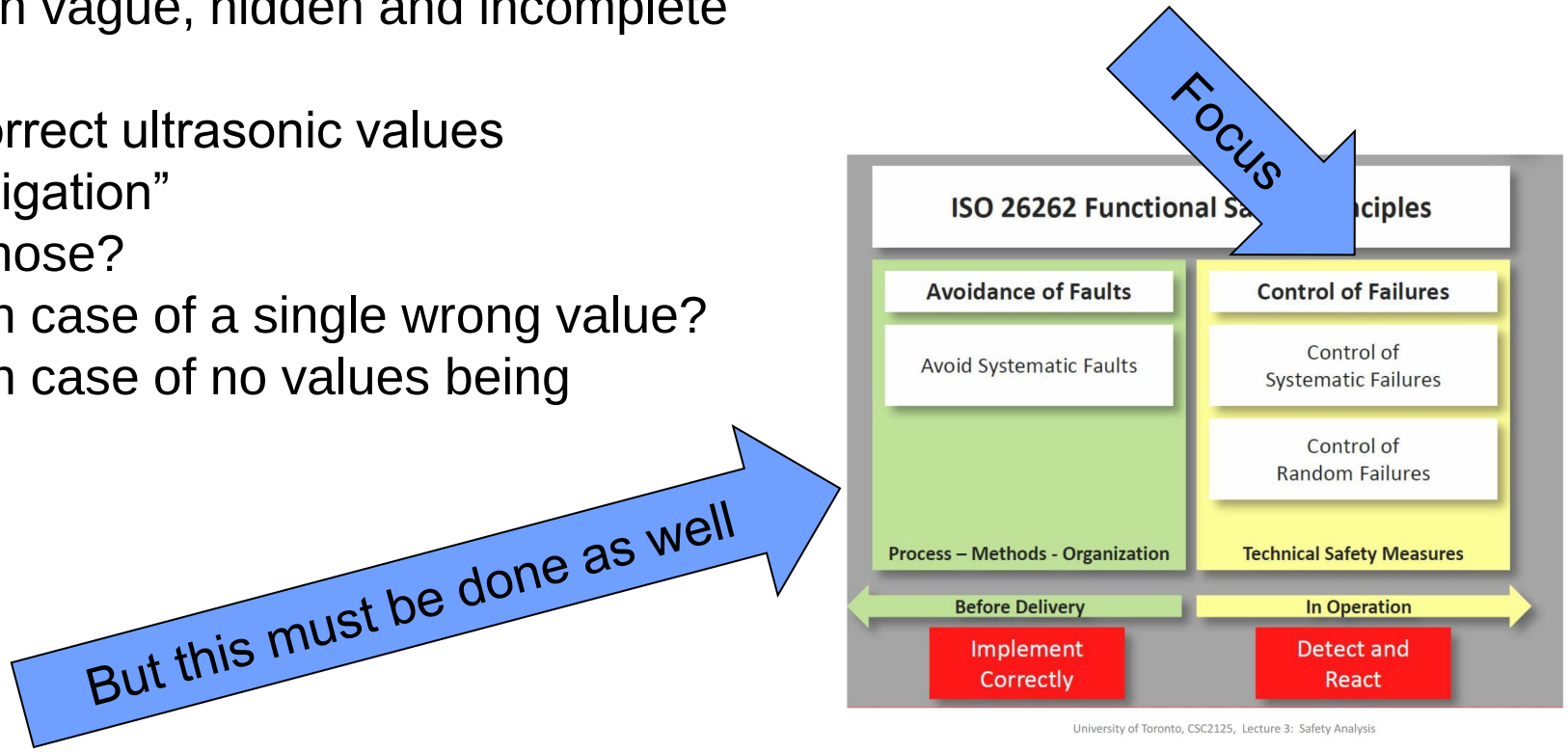


Developing an error handling architecture

- Obviously, the development of an error handling architecture requires a systematic process

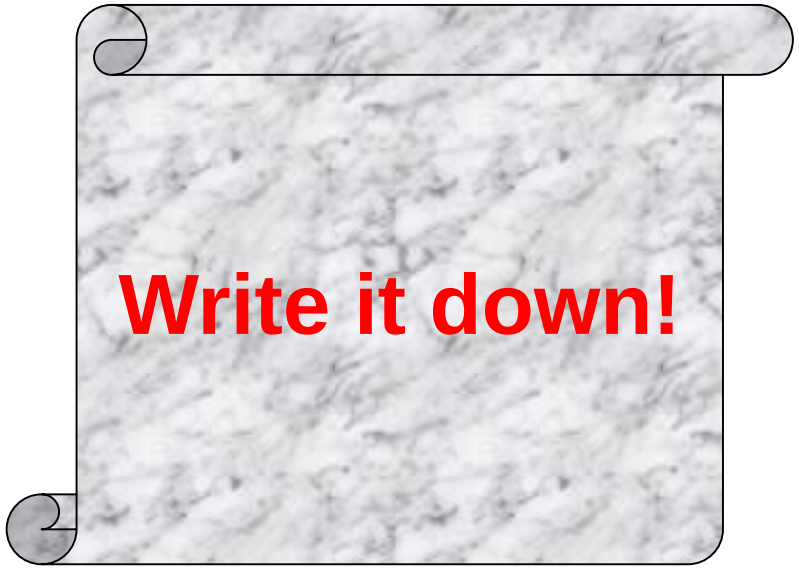
- Whereas functional requirements are often written down in the specification and can be implemented one by one…
- Error conditions are often vague, hidden and incomplete

- Example: “only 100% correct ultrasonic values may be used for the navigation”
  - How can we detect those?
  - What shall be done in case of a single wrong value?
  - What shall be done in case of no values being fired anymore?



Developing an error handling architecture

Step 1: Identify potential failures
Step 2: Check if these are critical and must be handled (default: YES)
Step 3: Identify, how these failures can be detected BEFORE they cause a real problem
Step 4: Decide, how the system shall react in case a fault / error / failure is detected (Safe State)
Step 5: Create an error handling architecture



Error Handling for our Car

Something, for which I didn't have the time when developing the first prototype

Functional Safety

Functional safety function failures may cause potential harm to humans, therefore these must be considered with high priority

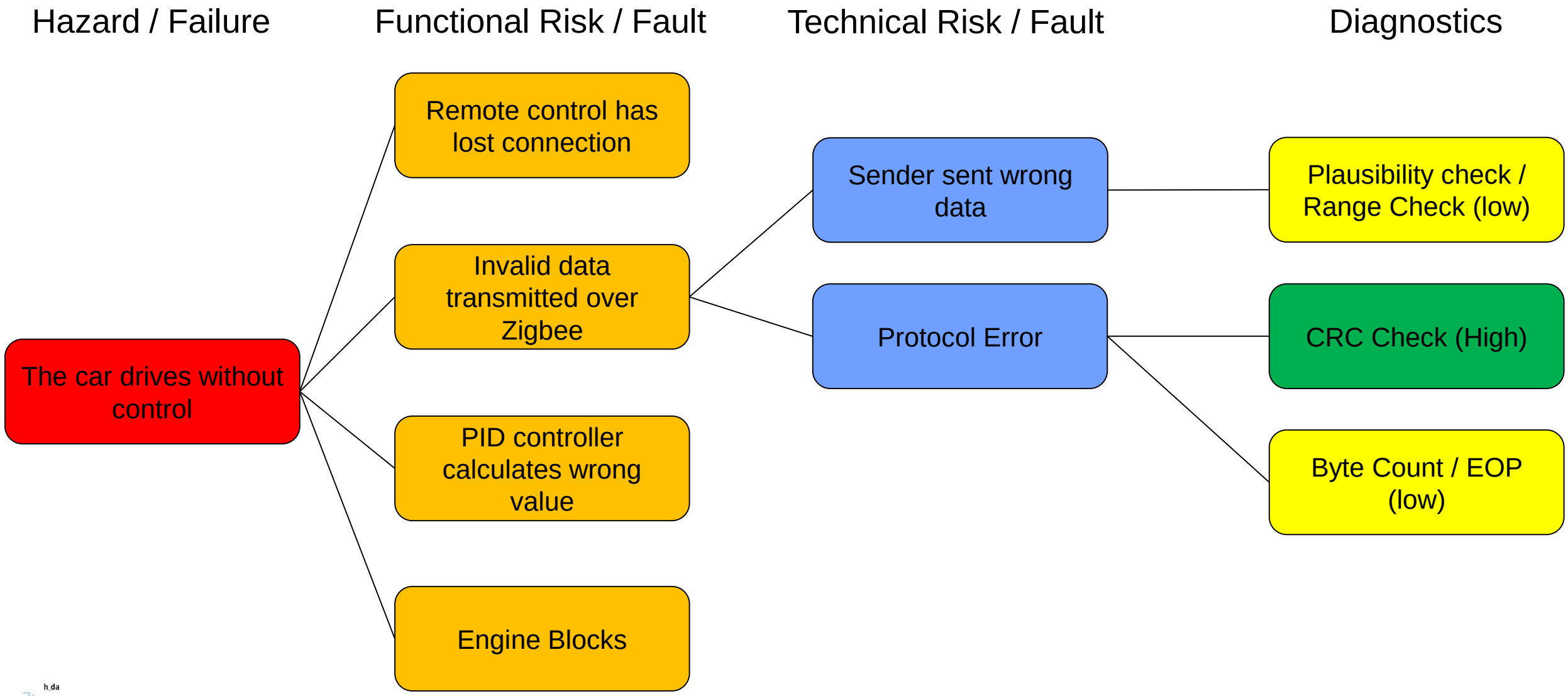
- The car shall not drive without control (safe system function)
  - Remote control has lost connection
  - Invalid data transmitted over XBee
  - Closed loop controller calculates invalid values or blocking engine

Diagnose and handle

- The car shall not bump against an obstacle (additional protection function)
  - Obstacles must be detected
  - The car must stop, no matter what the user or algorithm requests
- …

Additional function

Systematic Approach: Fault Tree Analysis (qualitative)



Possible Hardware Faults on the Car (incomplete)

Hardware can break / malfunction and typically suffers from environmental distortions

- Low Battery

- Xbee communication link too far distance / EMD distortions
- Ultrasonic may suffer from environmental distortions
- Lidar may suffer from sunlight (another car)
- Engine Bridge may overheat

Random but localised

- Microcontroller memory may have bitflips
- CPU may suffer from cosmic distortions
- Pins may be sticky

Random and system wide

- …

Possible Software Faults on the Car (incomplete)

Software error are typically caused by systematic design / programming faults, but may have a random behaviour

- Wrong reading / interpretation of Xbee protocol
- Wrong logic in engine controller

Systematic and localised

- Ignoring / not setting signal validity state and age
- Not checking sensor data for plausibility before using it
- Wrong program logic due to hardware failure (e.g. busy waiting for communication data or ADC sampling, missing interrupt for engine speed detection,…)

"Random" or state based but localised

- Uninitialized data
- Invalid pointers and/or memory allocation
- Realtime violations
- Races and Data inconsistencies
- …

Random and system wide

Error handling

- Some errors can be handled on application side
  - E.g. One Joystick protocol missed – let's take the old speed
- Some other errors should be handled more centrally
  - E.g. the signal storing the algorithm is invalid, let's stop the car under all circumstances
- Transition into a safe state and back must be designed
  - How quickly do we enter the safe state
  - What are the conditions to return from a safe state?
- How about system and hardware errors
  - E.g. we might suffer from task starvation

Safe State

The safe state of a system is an operational state, which must be reached in case of a dangerous failure of a safety item.

Example:
“In case of a critical sequence of failed protocols, the car shall come to an immediate stop.”

Entering a safe state is quite a drastic operation. It typically conflicts with the intended availability of the system.

Fail Safe – typically reached by turning of the power!

Fail Operational – a subset of functionality still must be ensured!

Entering the Safe State

Entering the Safe State

Some fundamental design considerations

- Entering the safe state must be simple, fast and robust!
- We typically cannot rely on too many components still working.

- The safe state must be kept safe until explicitly unlocked – preferably only at one place in the system / code.

- We might have to consider different safe states
  - The system is still operational and a known critical component failed (e.g. Xbee link)
  - In this case the system may return into normal operation once the link is available again

  - Or we have an undetermined or permanent failure and the car should be stopped permanently (at least for the current power cycle)

Entering the Safe State (Software)

Pure software solutions are cheap, because they do not require additional hardware.

- They are very flexible
- But are they really reliable?

Option 1 – Set the targetspeed to 0 (Stop)

- PID Controller might not work
- Engine might be blocking
- Race – error handling sets the targetspeed to 0, the normal application sets it to a non-zero value

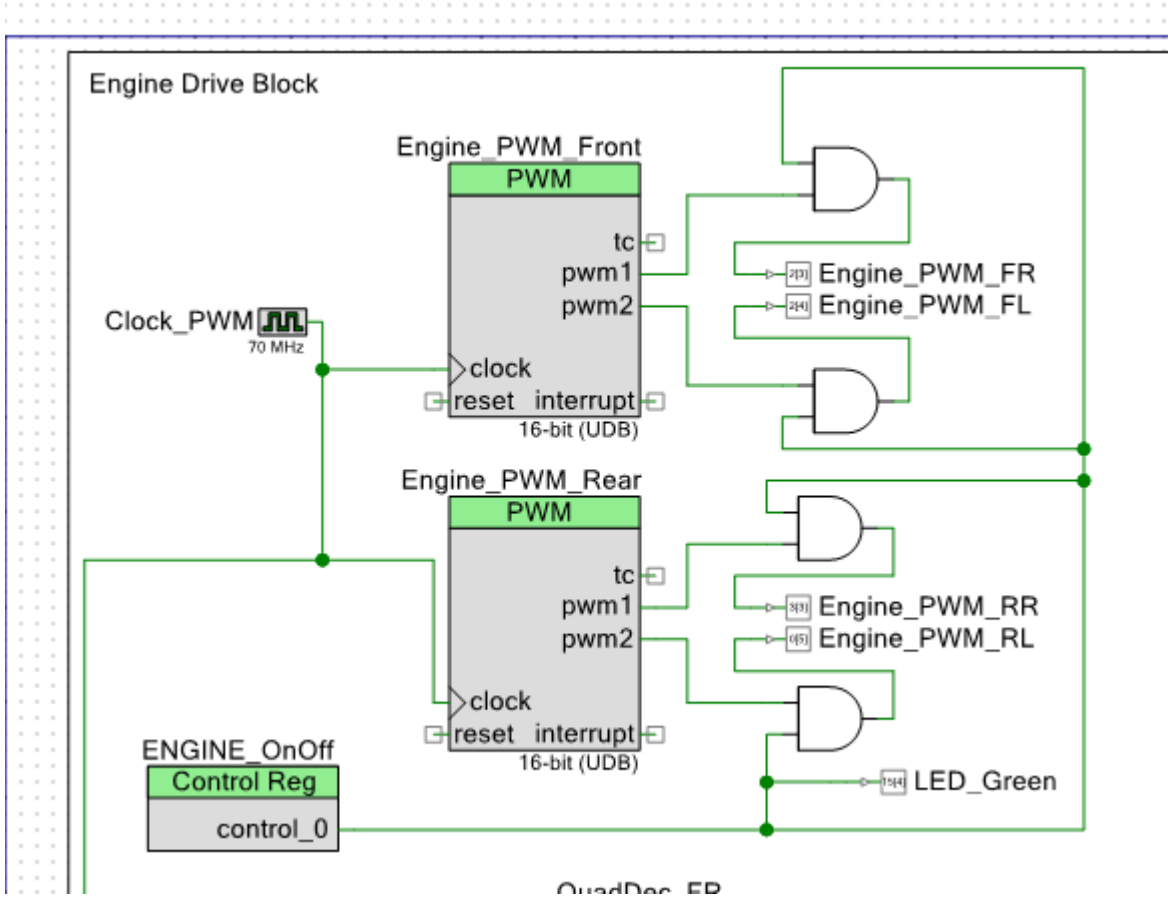
Option 2 – Set the PWM value or control registers to 0 (Stop)

- More robust, as PID controller logic does not affect the behaviour
- Blocking engine does not affect the behaviour
- Race risk persists

Entering the Safe State (Hardware 1)

Adding a logical AND gate to the PWM signals

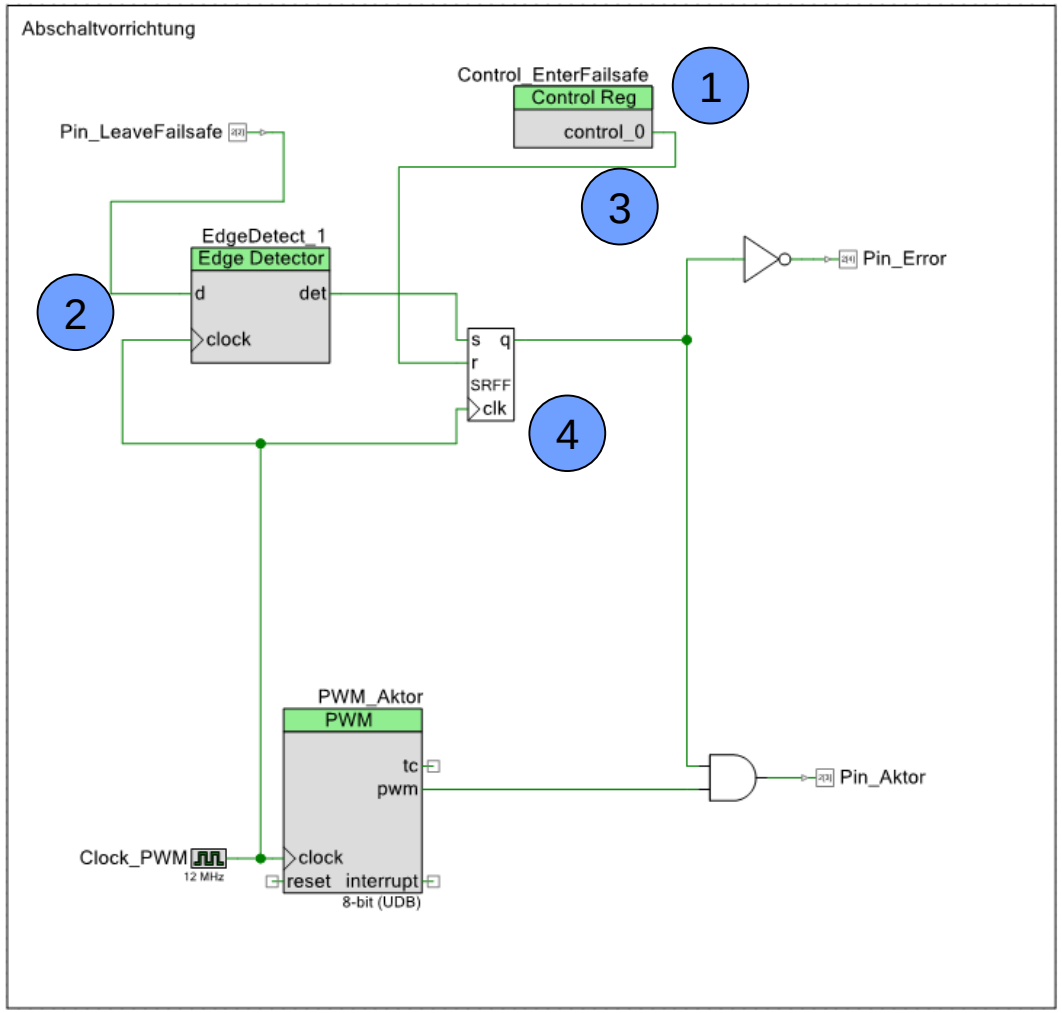
- All engines are commonly activated or deactivated
- The activation logic is separated from the drive logic (less risk for races)
- Additional hardware element (LED) can be connected to show the hardware state.
- Still, an unintended (Software) setting of ENGINE_OnOff can cause a problem.



Entering the Safe State (Hardware 2)

Adding a logical AND gate and a
User-Button Edge Detection / FlipFlop

- (1) A software controlled signal will enter the save state.
- (2) A user button press or a software controlled edge will leave the save state (transition required – reducing sticky pin risk)
- (3) but only if the software controlled signal is Off again.
- (4) a FlipFlop will keep the safe state until a valid unlock sequence appeared

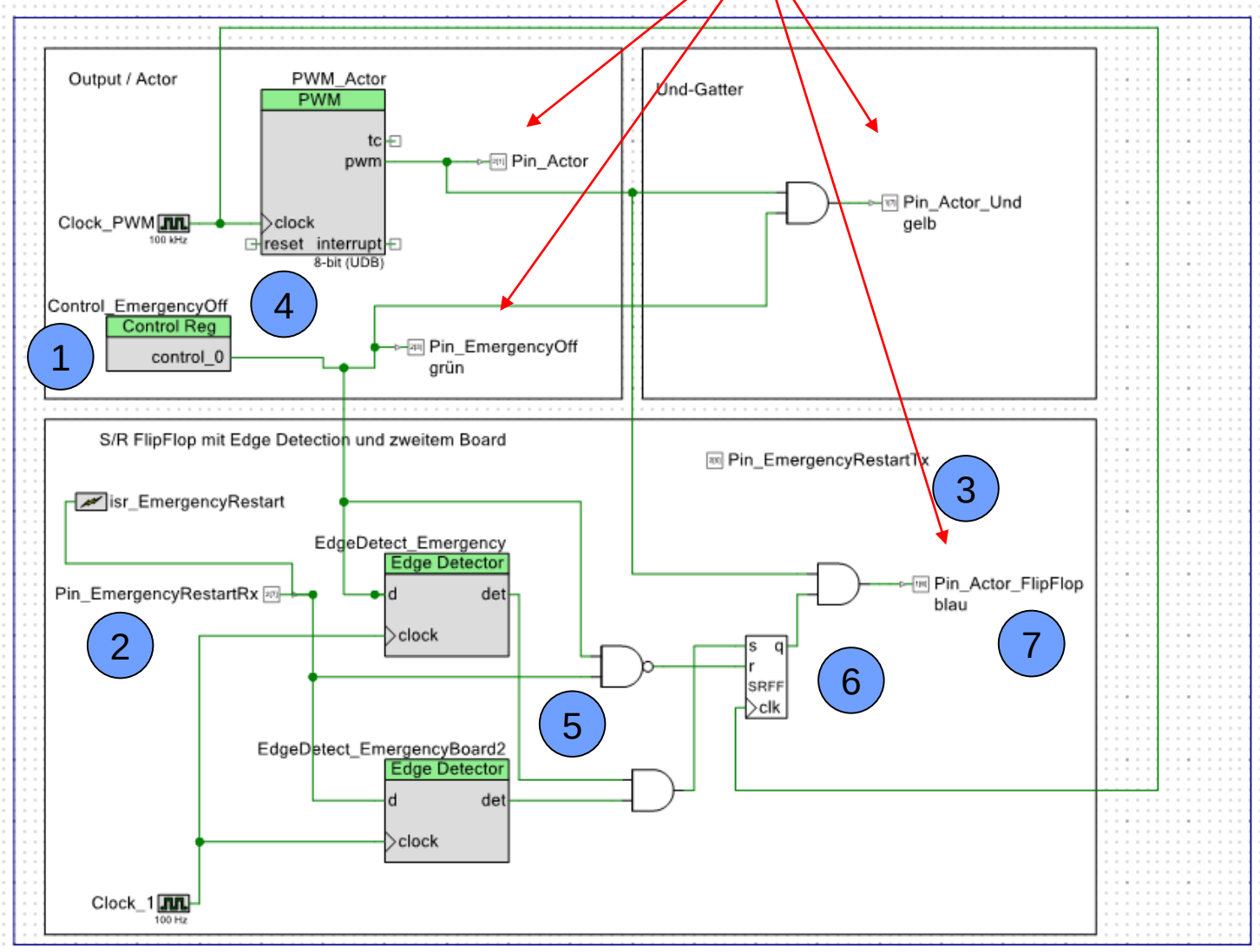


Entering the Safe State (Hardware 3)

Lot's of diagnostics ;-)

Using a button and checking the functionality of two redundant MCU before restart

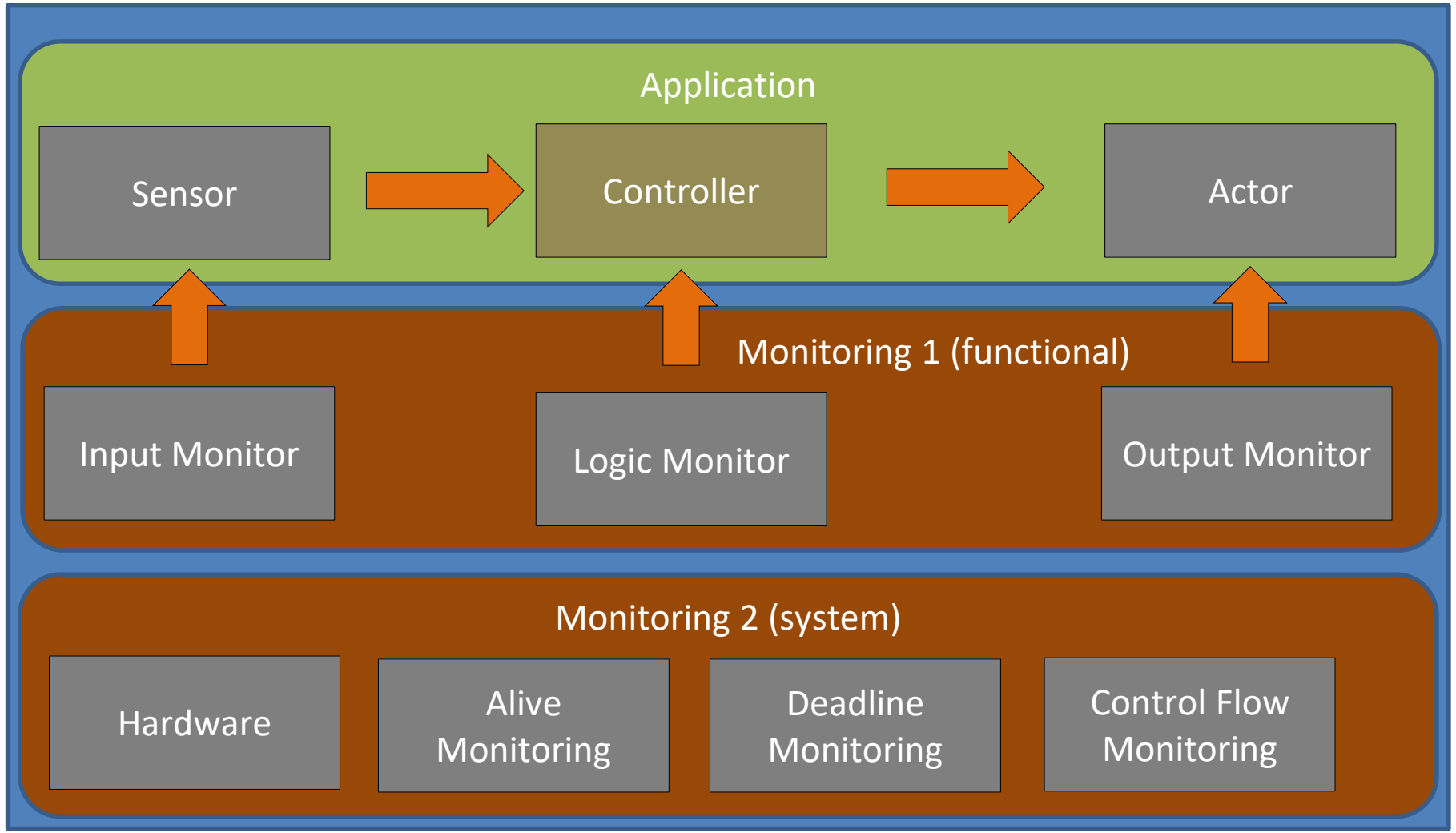
- (1) Emergency off
- (2) Button triggers ISR which
- (3) sets a pin which is connected to the second MCU
- (4) which resets the EmergencyOff register
- (5) Both edge detections have to detect the edge in the same cycle
- (6) resetting the RS FlipFlop
- (7) driving the PWM signal to the actor



Solution comparison

Solution	Reliability (enter)	SW Robustness	Hardware Robustness	Evaluation
SW 1	Low	Low	n.a.	Not ok
SW 2	Medium	Low	n.a.	May be acceptable for QM
HW 1	High	Medium (separated drive/stop)	Low (sticky pin)	Best price, ok for limited criticality (QM, ASIL A)
HW 2	High	Medium / High (separated drive/stop and pins)	Medium (different pins, edge detection)	Good reliability, robustness and availability
HW 3	High	High (Software must be executed to leave save state)	High (different pins, edge detection, cycle window)	Might be a bit too complex and cause availability issues.

Local versus Central Error Handling / Diagnostics



Very local, non - critical errors.

Critical errors, triggering a central error escalation

System errors, resulting in hardware handling (e.g. reset)

Implementation of a simple central monitor

List of monitors

```
//============================================================

const MONITOR_FctPtr_t MONITOR_ActiveCheck[] = {
    MONITOR__checkConnectionState,
    MONITOR__checkRemote,
    MONITOR__checkEngine,
    MONITOR__checkColission,
};

const uint16_t MONITOR_ActiveCheck_Size = sizeof(MONITOR_ActiveCheck)/sizeof(MONITOR_FctPtr_t);


/* USER CODE END SWC_MONITOR_USERDEFINITIONS */
```

Example monitor

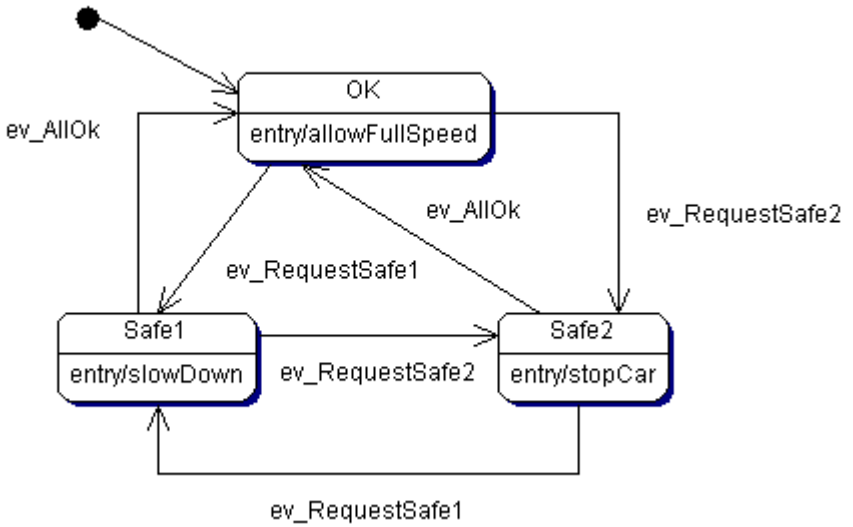
```
/**
 * Check if the remote control is active by checking the joystick signal age.
 **/
MONITOR_request_t MONITOR__checkRemote()
{
    uint32_t age = RTE_SC_JOYSTICK_getAge(&SO_JOYSTICK_signal);

    if (age > 3000) return MONITOR_safe_2;
    if (age > 500) return MONITOR_safe_1;
}
```

```
/*
 * component: swc_monitor
 * cycletime: 10
 * description: Cyclic runnable checking the system health
 * events:
 * name: MONITOR_health_run
 * shortname: health
 * signalIN: so_joystick|so_targetspeed|so_currentspeed|so_controlspeed|so_protocolRx|so_protocolTx|so_carState|so
 * signalOUT: so_carErrorState
 * task: tsk_io
 */
void MONITOR_health_run(RTE_event ev){

    /* USER CODE START MONITOR_health_run */

    SC_CARERRORSTATE_data_t carErrorState = SC_CARERRORSTATE_INIT_DATA;

    //Run all checker functions and escalate problem report
    MONITOR_request_t checkResult = MONITOR_allok;

    for (uint16_t i = 0; i < MONITOR_ActiveCheck_Size; i++)
    {
        MONITOR_request_t localCheck = MONITOR_ActiveCheck[i]();
        if (localCheck > checkResult) checkResult = localCheck;
    }

    //Some simple handling
    if (MONITOR_allok == checkResult)
    {
        carErrorState.m_led = ERROR_LED_NONE;
        carErrorState.m_maxSpeed = 100;
    }
    else if (MONITOR_safe_1 == checkResult)
    {
        carErrorState.m_led = ERROR_LED_YELLOW;
        carErrorState.m_maxSpeed = 50;
    }
    else  // Safe 2 oder Safe 3 - TODO create real state machine later
    {
        carErrorState.m_led = ERROR_LED_RED;
        carErrorState.m_maxSpeed = 0;
    }

    //Write to RTE and Driver
    RTE_SC_CARERRORSTATE_set(&SO_CARERRORSTATE_signal, carErrorState);
    RTE_SC_CARERRORSTATE_pushPort(&SO_CARERRORSTATE_signal);


    /* USER CODE END MONITOR_health_run */
}
```

Find most critical fault

Handle most critical fault

Report to RTE



Fault Tolerant Time Interval

ISO 26262-1:2018(E)

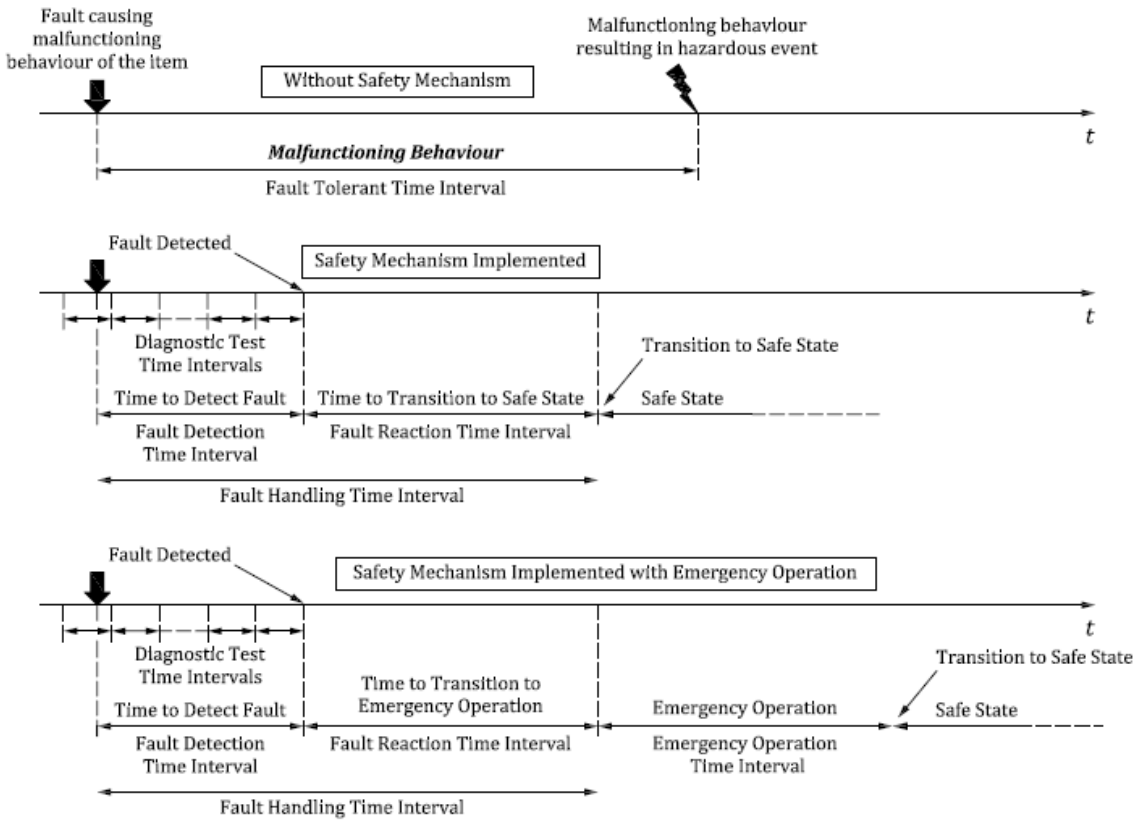
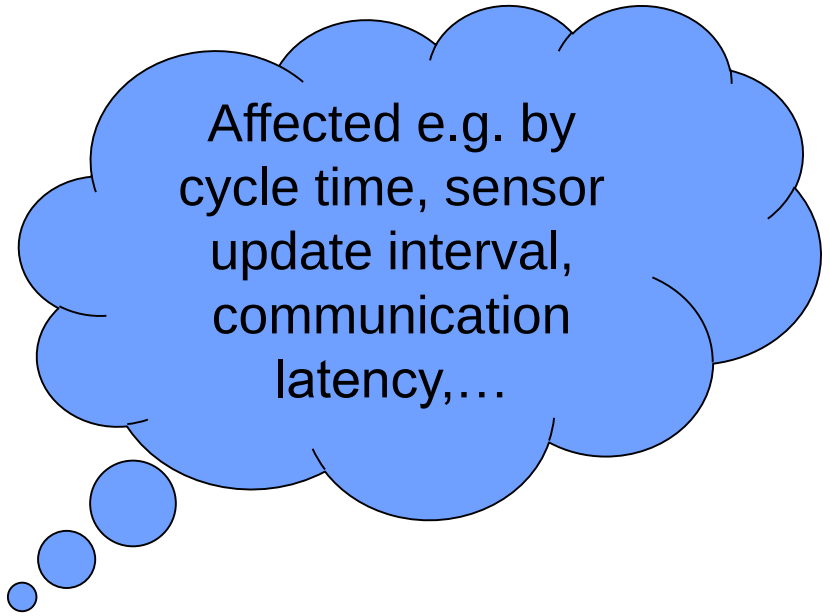


Figure 5 — Safety relevant time intervals

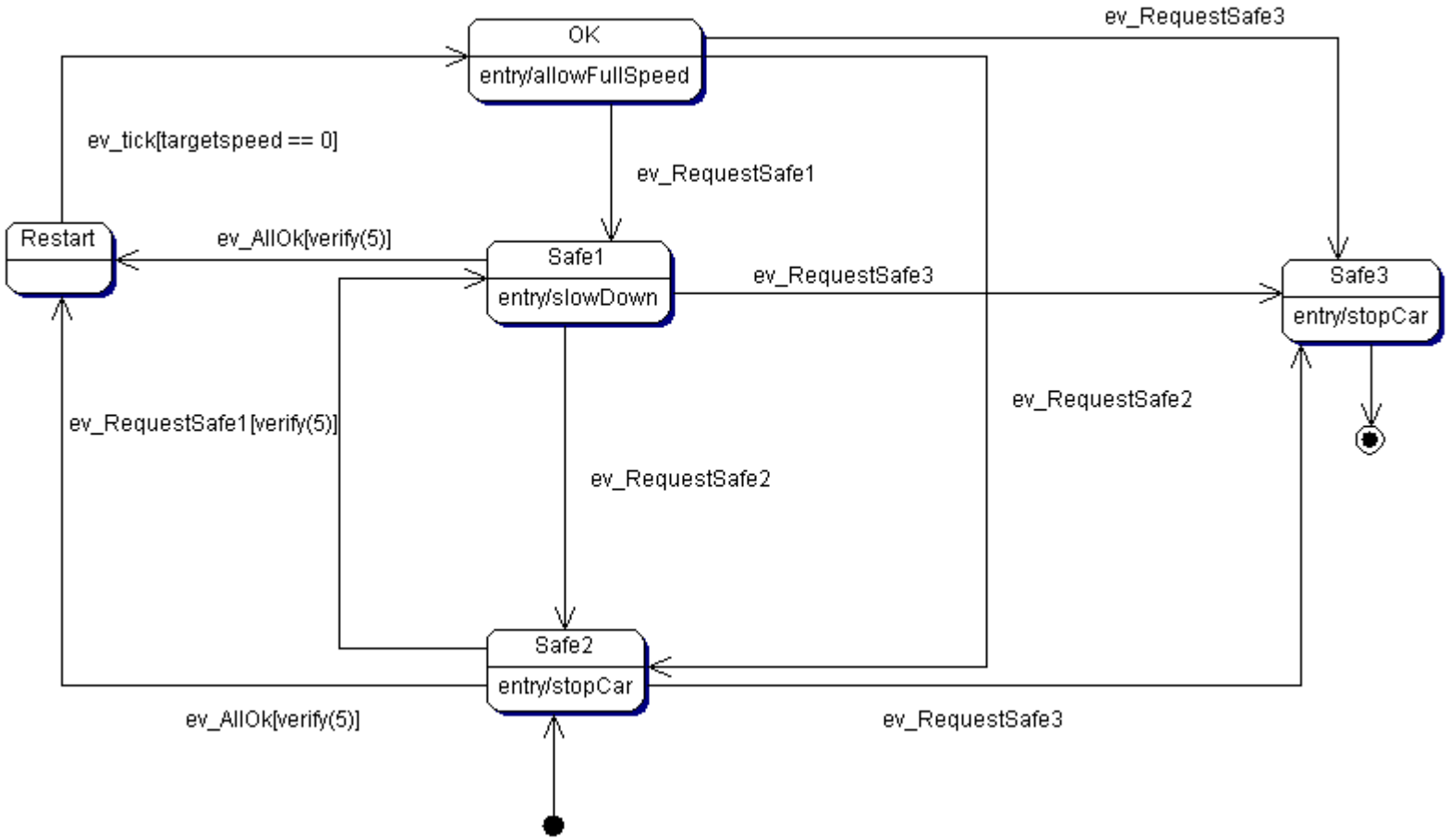


Extended Concept of an Error State Machine

Let’s consider the following additional requirements for our safety monitor:

- After entering the stop state, the joystick (or targetspeed) first needs to be in the middle (stop) position, before the car starts to drive again.
- Furthermore we want to verify a de-escalation for at least 5 cycles.
- In addition to the Safe2 StopState, we want to have a permanent StopState, which will not be left during the power cycle.
- We want to check all safety monitors before the car starts driving.

Extended Concept of an Error State Machine



Development Error Tracer

- The idea of a development error tracer is to record possible (unexpected) error conditions during development.
- I.e. it is more of a debugging tool.

Different solutions

- Using the UART Log (disadvantage – we need a wire)
- Using a toggling LED
- Using asserts

- Or use an internal error buffer which can be read out by a diagnostic command ➔ DET.

DET Motivation

Debugger
- Breakpoints
- Watch
- Instruction Trace

- Easy and flexible to use
- Limited data access (locals)
- Highly intrinsic

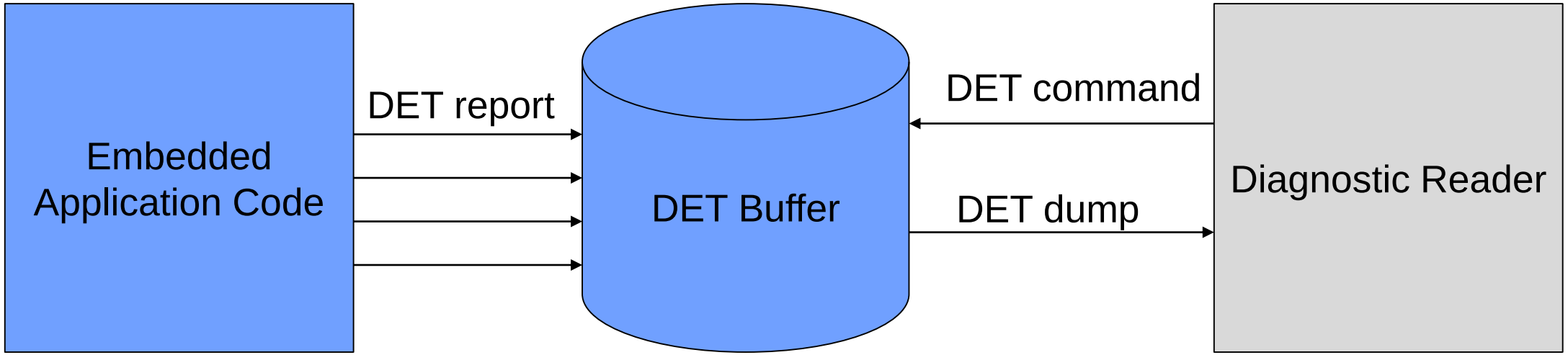
DET
- Data Storage
- Data dump
- Used in combination with debugger
- Starting point for field diagnostics

- Easy to use
- Stores any userdefined data
- Only limited by internal memory

Field Diagnostics
- Standardised Protocol
- “Garage Diagnostics”

- More complex, data protection
- Stores only relevant error codes
- Typically stored in non-volatile memory

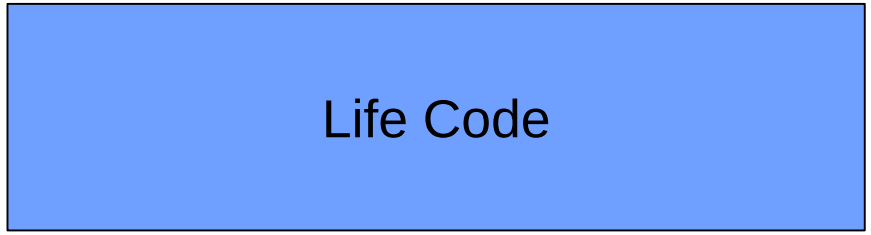
Example of a DET implementation



Example of a DET implementation / Challenges

- We do not have too much memory available on the embedded target ➔ store the data as compressed as possible
- The dump function will translate the data into a human readable format
- Idea: Use an SD card memory!

- As the report function will be called from different task contexts, we have to consider threadsafety requirements
  - Either use OS messages if available
  - Or lock the access using a semaphore



Wrap-Up

- Designing good error handling is as much effort as the normal development!
- It requires a clear analysis of possible (dangerous) failures, detection and handling concepts.

- Logging and Development Error Tracing functions help in analyzing the implemented behavior and to detect systematic faults.
- DET later on can be transferred into a real logging function